FXA REINVENT Project 2
Windows Command and Troubleshooting Record

REINVENT-based generative design of Factor Xa inhibitors using scaffold-aware ML scoring

Updated with final consistency checks, local Windows patch caveat, and 06B/06C reproducibility polish.

Important: Do not paste the PowerShell prompt itself. Paste only the commands shown in code blocks. The prompt text such as (fxa_reinvent4_py311) PS ...> is not part of a command.

1. Final Verified Project 2 Result

Metric	Original Prior	FXA TL Model	Rate-normalized interpretation
Valid molecules	982	911	Both scored sets were fully RDKit-valid after REINVENT invalid-SMILES removal.
Mean predicted pKi	6.531	6.674	+0.143 pKi shift.
Top-10 mean predicted pKi	7.573	8.021	+0.448 pKi improvement.
Max predicted pKi	7.921826	8.941557	+1.019731 pKi improvement.
Predicted pKi >= 7	102 / 982 = 10.4%	171 / 911 = 18.8%	+8.4 percentage-point enrichment.
Predicted pKi >= 8	0 / 982 = 0.0%	4 / 911 = 0.4%	TL generated high-scoring candidates absent from prior.
Basic druglike molecules	791 / 982 = 80.5%	600 / 911 = 65.9%	-14.7 percentage-point druglike trade-off.

Defensible conclusion: REINVENT transfer learning increased the fraction of generated molecules with predicted FXA pKi >= 7 from 10.4% to 18.8%, produced 4 predicted pKi >= 8 molecules compared with none from the prior, and improved the top-10 mean predicted pKi from 7.573 to 8.021. The druglike fraction decreased from 80.5% to 65.9%, so downstream filtering and docking validation are required.

2. Environment and Installation Commands That Worked

2.1 Create and activate the Python 3.11 environment

```
conda create -n fxa_reinvent4_py311 python=3.11 -y
conda activate fxa_reinvent4_py311
```

Why this was needed: The first Python 3.10 environment failed because REINVENT 4.8.24 required Python >= 3.11. The working environment was fxa_reinvent4_py311.

2.2 Install Git if missing

```
conda install -c conda-forge git -y
```

Troubleshooting: The first clone/install attempt failed because git was not recognized. Installing Git into the conda environment fixed this.

2.3 Install REINVENT4 from cloned source

```
cd C:\Users\eluma\Documents\mission_30days_3_portfolios\FXA_REINVENT_Portfolio\external\REINVENT4
python install.py cpu *> ..\..\scripts\02_reinvent_install_py311_retry2.log
Get-Content ..\..\scripts\02_reinvent_install_py311_retry2.log -Tail 120
```

Successful evidence: The successful install log showed reinvent-4.8.24 and a CPU PyTorch build installed in the fxa_reinvent4_py311 environment.

2.4 Download and verify the REINVENT prior

```
cd C:\Users\eluma\Documents\mission_30days_3_portfolios\FXA_REINVENT_Portfolio
New-Item -ItemType Directory -Force .\external\REINVENT4\priors
curl.exe -L "https://zenodo.org/records/15641297/files/reinvent.prior?download=1" -o ".
\external\REINVENT4\priors\reinvent.prior"
Test-Path .\external\REINVENT4\priors\reinvent.prior
Get-Item .\external\REINVENT4\priors\reinvent.prior | Select-Object Name,Length,LastWriteTime
```

Prior hash warning: During REINVENT runs, the log printed an invalid hash warning for reinvent.prior. This did not block loading or training. The real blocker later was unsupported tokens/elements in the TL seed file, not the hash warning.

2.5 Fix Pillow/RDKit dependency issues after install cleanup

```
python -m pip uninstall -y pillow PIL
conda remove pillow -y
python -m pip install --no-cache-dir --force-reinstall pillow
python -m pip install --no-cache-dir --only-binary=:all: "rdkit>=2025.09.1" "matplotlib>=3.7,<4"
python -m pip install charset-normalizer
python -m pip check
```

Troubleshooting: This fixed the PIL/Pillow DLL issue and missing dependencies after the REINVENT installation adjustments.

2.6 Windows-only local patch for REINVENT hw_report.py

```
notepad $env:CONDA_PREFIX\Lib\site-packages\reinvent\utils\hw_report.py
```

Local non-portable patch caveat: This fix edits an installed REINVENT package file inside the local conda environment. On a clean Windows environment, this patch must be re-applied, or REINVENT can be run under WSL/Linux where the Unix resource module is available. This patch made the Unix-only resource module optional so REINVENT could run on Windows.

Why this matters: The original error was ModuleNotFoundError: No module named resource. Documenting this patch prevents future confusion when reproducing on a fresh Windows install.

3. Project Setup Commands

3.1 Go to project folder and activate environment

```
cd C:\Users\eluma\Documents\mission_30days_3_portfolios\FXA_REINVENT_Portfolio
conda activate fxa_reinvent4_py311
```

3.2 Copy Project 1 model and reference data

```
Copy-Item ..\FXA_GNN_Portfolio\models\fxa_05b_best_scaffold_feature_model.joblib .\models\.
Copy-Item ..\FXA_GNN_Portfolio\data\processed\fxa_04_modeling_dataset.csv .\data\reference\.
```

3.3 Confirm REINVENT prior exists

```
Test-Path .\external\REINVENT4\priors\reinvent.prior
```

Expected output: True

4. Main Working Pipeline Commands

4.1 Step 01 - prepare potent FXA seed SMILES

```
python -m py_compile .\scripts\01_prepare_reinvent_seed_smiles.py
python .\scripts\01_prepare_reinvent_seed_smiles.py *> .\scripts\01.log
Get-Content .\scripts\01.log -Tail 120
```

Record: Successful output included 1249 potent unique seeds passing pKi >= 8.0 and STEP 01 COMPLETE.

4.2 Step 03 - prepare flat stereo-free TL SMILES

```
python -m py_compile .\scripts\03_prepare_tl_flat_seed_smiles.py
python .\scripts\03_prepare_tl_flat_seed_smiles.py *> .\scripts\03.log
Get-Content .\scripts\03.log -Tail 120
```

Record: Successful output included 1211 unique all-flat SMILES, 1017 druglike flat SMILES, zero stereo tokens, and STEP 03 COMPLETE.

4.3 Step 04B - filter by REINVENT-supported elements

```
python -m py_compile .\scripts\04b_filter_tl_smiles_by_reinvent_supported_elements.py
python .\scripts\04b_filter_tl_smiles_by_reinvent_supported_elements.py *> .\scripts\04b.log
Get-Content .\scripts\04b.log -Tail 120
```

Record: Successful output: 1211 input SMILES, 1199 supported SMILES, 12 removed, unsupported element counts B=9, I=2, P=1, STEP 04B COMPLETE.

4.4 Step 04A - split filtered all-flat SMILES into train/validation

```
python -m py_compile .\scripts\04a_make_tl_train_valid_split.py
python .\scripts\04a_make_tl_train_valid_split.py *> .\scripts\04a.log
Get-Content .\scripts\04a.log -Tail 120
```

Record: Final intended route: all-flat elements file. Expected counts: 1199 input, 959 train, 240 validation, zero train/validation overlap, STEP 04A COMPLETE.

4.5 Verify final TL train/validation files

```
Test-Path .\data\reference\tl_split\fxa_tl_train_pki8_all_flat_elements.smi
Test-Path .\data\reference\tl_split\fxa_tl_valid_pki8_all_flat_elements.smi
Get-Content .\data\reference\tl_split\fxa_tl_train_pki8_all_flat_elements.smi | Measure-Object -Line
Get-Content .\data\reference\tl_split\fxa_tl_valid_pki8_all_flat_elements.smi | Measure-Object -Line
```

Record: Expected: True, True, 959, 240.

4.6 Step 04 - run REINVENT transfer learning

```
reinvent -l .\scripts\04.log .\configs\04_tl_fxa_pki8_all_flat_sanity.toml
Get-Content .\scripts\04.log -Tail 180
```

Record: Successful output included Best validation loss at epoch 5 and Finished REINVENT.

4.7 Confirm TL model files

```
Get-ChildItem .\models -Filter "fxa_reinvent_tl_pki8_all_flat_sanity.model*"
Get-ChildItem .\models | Sort-Object LastWriteTime -Descending | Select-Object -First 10 Name,Length,LastWriteTime
```

Record: Successful files included the final .model plus epoch 1-5 .chkpt files.

4.8 Step 05 - sample 1000 molecules from TL model

```
reinvent -l .\scripts\05.log .\configs\05_sample_tl_fxa_pki8_all_flat_sanity_1000.toml
Get-Content .\scripts\05.log -Tail 120
Import-Csv .\data\generated\reinvent_tl_fxa_pki8_all_flat_sanity_1000.csv | Select-Object -First 10
```

Record: Output file: data/generated/reinvent_tl_fxa_pki8_all_flat_sanity_1000.csv.

4.9 Step 05D - score TL-generated molecules

```
python -m py_compile .\scripts\05_score_tl_generated_smiles_with_fxa_model.py
python .\scripts\05_score_tl_generated_smiles_with_fxa_model.py *> .\scripts\05_score.log
Get-Content .\scripts\05_score.log -Tail 160
```

Record: Successful output: 911 valid molecules, 600 basic druglike, mean pKi 6.673987780979443, max pKi 8.941557428836823, 171 molecules >=7, 4 molecules >=8.

4.10 Step 06A - sample 1000 molecules from original prior

```
reinvent -l .\scripts\06_sample_prior.log .\configs\06_sample_prior_1000.toml
Get-Content .\scripts\06_sample_prior.log -Tail 120
Import-Csv .\data\generated\reinvent_prior_1000.csv | Select-Object -First 10
```

Record: Output file: data/generated/reinvent_prior_1000.csv.

4.11 Step 06B - score prior-generated molecules

```
python -m py_compile .\scripts\06_score_prior_generated_smiles_with_fxa_model.py
python .\scripts\06_score_prior_generated_smiles_with_fxa_model.py *> .\scripts\06_score_prior.log
Get-Content .\scripts\06_score_prior.log -Tail 160
```

Record: Source-of-truth prior values: 982 valid, 791 basic druglike, mean pKi 6.531452, max pKi 7.921826, 102 molecules >=7, 0 molecules >=8.

4.12 Step 06C - compare prior vs TL

```
python -m py_compile .\scripts\06_compare_prior_vs_tl_scores.py
python .\scripts\06_compare_prior_vs_tl_scores.py *> .\scripts\06_compare.log
Get-Content .\scripts\06_compare.log -Tail 120
```

Record: Successful comparison: mean pKi shift +0.143, top-10 mean pKi shift +0.448, pKi >=7 count shift +69, pKi >=8 count shift +4.

4.13 Final source-of-truth comparison check

```
python .\scripts\06_compare_prior_vs_tl_scores.py *> .\scripts\06_compare_final_check.log
Get-Content .\scripts\06_compare_final_check.log -Tail 120
```

Record: Verified final values: prior/TL valid 982/911, pKi >=7 102/171, pKi >=8 0/4, druglike 791/600, max pKi 7.921826/8.941557.

4.14 Step 07 - prioritize TL candidates

```
python -m py_compile .\scripts\07_prioritize_tl_fxa_candidates.py
python .\scripts\07_prioritize_tl_fxa_candidates.py *> .\scripts\07.log
Get-Content .\scripts\07.log -Tail 160
```

Record: Successful output: 600 valid + novel + druglike candidates, 2 Tier 1, 13 Tier 2, 77 Tier 3, top-30 docking queue created.

4.15 Step 08 - create final figures and summary tables

```
python -m py_compile .\scripts\08_make_project2_summary_figures.py
python .\scripts\08_make_project2_summary_figures.py *> .\scripts\08.log
Get-Content .\scripts\08.log -Tail 160
```

Record: Successful output: STEP 08 COMPLETE.

5. Final Internal-Consistency Checks

5.1 Confirm final workflow uses all-flat elements, not druglike-flat training

```
Get-ChildItem .\configs, .\scripts -Recurse -File -Include *.toml,*.py |
Select-String -Pattern "druglike_flat" -CaseSensitive:$false
```

Interpretation: References to druglike_flat in comments, options, or old branching logic are acceptable. The active TL TOML should not point to druglike_flat training files. It should point to the all-flat elements train/validation files.

```
Select-String -Path .\configs\04_tl_fxa_pki8_all_flat_sanity.toml -Pattern "smiles_file|validation_smiles_file|
druglike_flat|all_flat_elements"
```

Expected active TL paths: smiles_file should be data/reference/tl_split/fxa_tl_train_pki8_all_flat_elements.smi and validation_smiles_file should be data/reference/tl_split/fxa_tl_valid_pki8_all_flat_elements.smi.

Why this check matters: Step 03 produced both an all-flat set of 1211 molecules and a druglike-flat set of 1017 molecules. The final TL route used the all-flat set, then filtered unsupported elements to 1199, then split to 959 train and 240 validation. The 1017 druglike-flat number is valid but was not the final TL training input.

5.2 Clarify 06B and 06C numbering

Reproducibility polish: Two script filenames begin with 06_: 06_score_prior_generated_smiles_with_fxa_model.py and 06_compare_prior_vs_tl_scores.py. This is harmless because each command is explicit, but in documentation they should be described as Step 06B and Step 06C. README and write-up script lists should use the same Step 06B/06C wording.

6. Troubleshooting Notes and What Fixed Each Issue

Issue	What happened	Fix / interpretation
Python version mismatch	Python 3.10 failed because REINVENT 4.8.24 required Python >= 3.11.	Created and used fxa_reinvent4_py311.
Git not recognized	The REINVENT source install workflow	Installed Git into the conda environment

	required Git.	with conda install -c conda-forge git -y.
Missing REINVENT prior	The cloned REINVENT repository did not include reinvent.prior.	Downloaded the official prior to external/REINVENT4/priors/reinvent.prior and verified the file existed.
Windows resource module error	REINVENT imported Unix-only resource module, which is unavailable on native Windows.	Applied a local Windows compatibility patch in hw_report.py to make resource optional. Caveat: this edits an installed package file and must be re-applied on a clean Windows environment, or use WSL/Linux where resource exists.
Pillow/PIL DLL issue	Pillow import failed after installation cleanup.	Uninstalled/reinstalled Pillow and restored RDKit/matplotlib/charset-normalizer dependencies.
Unsupported B token during TL	REINVENT prior vocabulary did not support a molecule containing boron.	Replaced brittle token allow-list idea with an RDKit element filter. Removed 12 molecules containing B, I, or P.
Token allow-list false-positive risk	Hard-coded tokens from a single error message could wrongly remove valid protonated or ring-closure SMILES.	Used RDKit atom-symbol filtering instead of SMILES token filtering.
Druglike vs all-flat split mixup	A rerun of Step 04A initially used druglike_flat instead of all_flat.	Patched INPUT_MODE to all_flat and used fxa_reinvent_tl_pki_ge_8p0_flat_reinvent_elements_smiles_only.smi.
REINVENT failed-to-sanitize messages	During TL monitoring, many sampled SMILES failed to sanitize.	Correctly treated these as non-fatal sampling-monitor warnings because the TL run finished successfully. Later sampling output was scored only after RDKit validation.
RF tree uncertainty with sklearn Pipeline	Individual RF trees cannot be called on raw 2048-bit features after the pipeline has a VarianceThreshold step.	Applied fitted preprocessing steps before per-tree predictions for RF uncertainty.
Unequal denominator comparison	Prior produced 982 valid molecules while TL produced 911, so raw counts alone were not defensible.	Added rate-normalized interpretation: pKi >=7 increased from 10.4% to 18.8%; druglike fraction decreased from 80.5% to 65.9%.

7. Final Output Files

```
results\tables\project2_final_key_metrics.csv
results\figures\prior_vs_tl_key_metrics.png
results\figures\prior_vs_tl_predicted_pki_distribution.png
results\figures\tl_candidate_tier_counts.png
results\metrics\project2_final_summary.json
results\metrics\project2_final_summary.txt
results\tables\tl_fxa_docking_queue_top30.csv
```

8. Optional Helper Commands

```
Get-ChildItem .\data\generated
Get-ChildItem .\results\tables
Get-ChildItem .\results\figures
Import-Csv .\results\tables\tl_fxa_docking_queue_top30.csv | Select-Object -First 10
Import-Csv .\results\tables\project2_final_key_metrics.csv
```

9. Final Learning Summary

This record documents a complete Windows-based Project 2 workflow: installing REINVENT4, resolving Windows-specific issues, preparing FXA seed molecules, running transfer learning, scoring generated molecules with a scaffold-aware FXA ML model, normalizing the prior-vs-TL comparison by valid molecule counts, and creating a final docking queue. The troubleshooting steps are part of the scientific record because they show how each failure mode was diagnosed and fixed.